

Razor Pages en .NET 10

Cómo una URL, HTML y una clase C# trabajan juntas

Razor Pages organiza la web alrededor de cada página

La idea

Cada pantalla reúne su ruta, su interfaz y la lógica que atiende las peticiones.

El código relacionado queda cerca y es fácil de localizar.

Dónde encaja especialmente bien

- Formularios
- Altas, consultas, cambios y borrados
- Aplicaciones centradas en páginas
- Proyectos educativos y administrativos

Cada página une ruta, interfaz y lógica

1 · Ruta

La ubicación dentro de Pages determina normalmente la URL.

2 · .cshtml

Contiene HTML, directivas Razor, Tag Helpers y datos del modelo.

3 · PageModel

Contiene propiedades y métodos C# como OnGet y OnPost.

Dos archivos colaboran para construir una página

Ejemplo: Pages/Categorias/Index

Por convención, el archivo de C# añade .cs al nombre del archivo Razor. @model conecta ambos.

El alumnado debe aprender a abrirlos juntos.

Index.cshtml

HTML y directivas Razor
@page y @model
Presentación de los datos

Index.cshtml.cs

Clase que hereda de PageModel
Propiedades para la página
Manejadores OnGet / OnPost

Una petición GET recorre tres pasos

PETICIÓN

C#

RESPUESTA

1 · Llega la URL

El enrutamiento encuentra el archivo dentro de Pages.

2 · Se ejecuta OnGet

El PageModel prepara los datos necesarios.

3 · Razor crea HTML

El servidor envía al navegador HTML, no código C#.

Program.cs activa y publica las Razor Pages

Configuración mínima en .NET 10

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddRazorPages();

var app = builder.Build();

app.UseHttpsRedirection();
app.UseRouting();
app.UseAuthorization();

app.MapStaticAssets();
app.MapRazorPages()
    .WithStaticAssets();

app.Run();
```

AddRazorPages registra los servicios

ASP.NET Core sabrá crear PageModel, aplicar binding, validación y demás servicios.

MapRazorPages añade las rutas. En .NET 10, los proyectos nuevos también integran la gestión optimizada de archivos estáticos.

Razor inserta valores C# dentro del HTML

Index.cshtml

```
@page
@model IndexModel

<h1>@Model.Titulo</h1>

<ul>
@foreach (Categoria categoria
         in Model.Categorias)
{
    <li>@categoria.Nombre</li>
}
</ul>
```

Cómo leerlo

- @page convierte el archivo en una página enrutable.
- @model indica qué PageModel utiliza.
- @ cambia temporalmente de HTML a C#.
- Model permite leer sus propiedades.

El PageModel prepara los datos y toma decisiones

Index.cshtml.cs

```
public class IndexModel(  
    TrivialContext contexto) : PageModel  
{  
    public List<Categoria> Categorias  
        { get; set; } = [];  
  
    public async Task OnGetAsync()  
    {  
        Categorias = await contexto.Categorias  
            .OrderBy(c => c.Nombre)  
            .ToListAsync();  
    }  
}
```

Responsabilidades

1. Recibir servicios por el constructor.
2. Consultar o modificar datos.
3. Exponer propiedades a .cshtml.
4. Devolver Page, RedirectToPage, NotFound u otro resultado cuando sea necesario.

GET muestra información; POST procesa cambios

Los nombres de los manejadores siguen una convención

OnGet / OnGetAsync responden a GET. OnPost / OnPostAsync responden a formularios enviados con POST.

El sufijo Async acompaña a métodos asíncronos.

OnGetAsync

Abre una página

Carga datos

No debería modificar información

OnPostAsync

Recibe un formulario

Valida y guarda

Suele terminar con una redirección

El model binding convierte datos HTTP en valores C#

Ruta

@page "{id:int}" → OnGetAsync(int id)

Query string

?busqueda=redes → string? busqueda

Formulario

asp-for genera nombres que ASP.NET Core puede enlazar.

PageModel

[BindProperty] permite recibir un objeto mediante POST.

[BindProperty] conecta el formulario con una propiedad

Editor.cshtml

```
@page "{id:int}"
@model EditorModel

<form method="post">
    <input asp-for="Categoria.Id"
        type="hidden" />

    <label asp-for="Categoria.Nombre">
    </label>
    <input asp-for="Categoria.Nombre"
        class="form-control" />

    <span asp-validation-for=
        "Categoria.Nombre"></span>

    <button type="submit">Guardar</button>
</form>
```

Editor.cshtml.cs

```
[BindProperty]
public Categoria Categoria
    { get; set; } = new();

public async Task<IActionResult>
    OnPostAsync()
{
    if (!ModelState.IsValid)
        return Page();

    contexto.Categorias.Update(Categoria);
    await contexto.SaveChangesAsync();
    return RedirectToPage("Index");
}
```

Los nombres generados por asp-for coinciden con la ruta de la ¹¹ propiedad.

La validación detiene el guardado si los datos no son válidos

1 · Reglas

```
[Required]  
[StringLength(50)]  
public string Nombre { get; set; } = "";
```

3 · Comprobación

```
if (!ModelState.IsValid)  
    return Page();
```

2 · Binding

ASP.NET Core crea el objeto y registra los errores.

4 · Mensajes

asp-validation-for muestra el error junto al campo.

La DI entrega servicios al PageModel

1 · Registrar

```
builder.Services  
    .AddDbContext<TrivialContext>();
```

2 · Recibir

```
public class IndexModel(  
    TrivialContext contexto)  
    : PageModel
```

3 · Utilizar

```
await contexto.Categorias  
    .ToListAsync();
```

Async y await evitan bloquear el servidor

Operaciones habituales

```
Categoria? categoria =  
    await contexto.Categorias  
        .FindAsync(id);
```

```
List<Categoria> categorias =  
    await contexto.Categorias  
        .ToListAsync();
```

```
await contexto.SaveChangesAsync();
```

Qué significa realmente

Mientras la aplicación espera una operación de entrada/salida, el servidor puede atender otras solicitudes.

No hace que la consulta SQL sea mágicamente más rápida: mejora el uso de los recursos del servidor.

Un CRUD se divide en páginas pequeñas y predecibles

Index

GET consulta y muestra el listado.

Crear

GET muestra el formulario. POST valida y añade.

Editar

GET carga por id. POST valida y actualiza.

Eliminar

POST localiza y borra tras confirmar la acción.

Una página de edición repite siempre el mismo patrón

Editar.cshtml.cs

```
public async Task<IActionResult>
    OnGetAsync(int id)
{
    Categoria? categoria =
        await contexto.Categorias.FindAsync(id);
    if (categoria is null) return NotFound();
    Categoria = categoria;
    return Page();
}

public async Task<IActionResult>
    OnPostAsync()
{
    if (!ModelState.IsValid) return Page();
    contexto.Categorias.Update(Categoria);
    await contexto.SaveChangesAsync();
    return RedirectToPage("Index");
}
```

Cómo razonarlo

GET

1. Recibe el id.
2. Busca la entidad.
3. Devuelve 404 o muestra la página.

POST

1. Recibe el formulario.
2. Valida.
3. Guarda.
4. Redirige al listado.

Reglas para un proyecto claro y seguro

Buenas prácticas para las primeras aplicaciones

El objetivo no es escribir menos líneas a cualquier precio, sino que cada línea tenga una responsabilidad comprensible. La simplicidad también evita errores.

Conviene

- HTML sencillo; lógica en PageModel
- Validar antes de guardar
- RedirectToPage tras un POST correcto

Evita

- Consultar toda la tabla sin necesidad
- Enlazar propiedades sensibles
- Mezclar acceso a datos dentro del HTML